

Dantzig-Wolfe decomposition

Guillaume Dalle

MPRO – Decomposition methods for integer programming

All of the content is taken from [1] or [2].

Contents

I.	Motivation: complicating constraints	1
II.	Dantzig-Wolfe relaxation	4
II.1.	Definition	4
II.2.	Extended formulation	5
II.3.	Back to coupling constraints	5
II.4.	Examples	6
III.	Column generation	8
III.1.	Subsets of variables	8
III.2.	Pricing	10
III.3.	Full algorithm	11
III.4.	Examples	11
IV.	Branch and price	12
IV.1.	Branching techniques	12
IV.2.	Implementation tricks	13
IV.3.	Flow subproblems	13
IV.4.	Getting an integer solution	13
IV.5.	Take-home messages	14
	Bibliography	14

I. Motivation: complicating constraints

For each of the following problems:

1. Give a mixed integer programming formulation.
2. Describe the structure of the constraint matrix.

Generalized assignment – modeling

Exercise 1

There are a number of tasks $i \in [m]$ to be performed by agents $j \in [n]$. Assigning task i to agent j generates profit c_{ij} and takes time t_{ij} . Each agent has a time budget T_j . Agents do not collaborate on tasks. The goal is to maximize the profit subject to time constraints.

Let x_{ij} be a binary variable equal to 1 if task i is assigned to agent j . The GAP can be formulated as:

$$\max_x \sum_{i,j} c_{ij} x_{ij} \quad \text{s.t.} \quad \begin{cases} \sum_j x_{ij} \leq 1 & \forall i \\ \sum_i t_{ij} x_{ij} \leq T_j & \forall j \\ x_{ij} \in \{0, 1\} & \forall (i, j) \end{cases} \quad (1)$$

The constraint matrix has a block diagonal structure with one block per agent j and a coupling block on top (at most one agent per task).

Cutting stock – modeling

Exercise 2

At a paper mill, we have large rolls of paper of width W , to be cut into smaller rolls. The client's order is to produce b_i rolls of width w_i for each $i \in [m]$. The goal is to minimize the number of large rolls necessary.

We assume that at most p large rolls will be necessary. Let y_j be a binary variable equal to 1 if roll $j \in [p]$ is used, and z_{ij} be an integer variable denoting the number of small rolls of width w_i cut out from large roll j . The CSP can be formulated as:

$$\min_{y,z} \sum_j y_j \quad \text{s.t.} \quad \begin{cases} \sum_i w_i z_{ij} \leq W y_j & \forall j \\ \sum_j z_{ij} \geq b_i & \forall i \\ y_j \in \{0, 1\} & \forall j \\ z_{i,j} \in \mathbb{N} & \forall (i, j) \end{cases} \quad (2)$$

The constraint matrix has a block diagonal structure with one block per large roll j and a coupling block on top (demand satisfaction).

Facility location – modeling

Exercise 3

A company would like to set up facilities to serve its customers $i \in [m]$. For each possible facility location $j \in [n]$, the fixed annual operating cost is f_j and the cost of serving client i is c_{ij} . The goal is to select which facilities to open in order to minimize total annual costs.

Let x_j be a binary variable equal to 1 if facility j is open, and y_{ij} be the fraction of demand d_i transported from j to i . The FLP can be formulated as:

$$\min_{x,y} \sum_j f_j x_j + \sum_{i,j} c_{ij} y_{ij} \quad \text{s.t.} \quad \begin{cases} \sum_j y_{ij} = 1 & \forall i \\ y_{ij} \leq x_j & \forall (i, j) \\ x_j \in \{0, 1\} & \forall j \\ y_{i,j} \geq 0 & \forall (i, j) \end{cases} \quad (3)$$

The constraint matrix has a block diagonal structure with one block per facility j and a coupling block on top (serving clients).

Network design – modeling

Exercise 4

Let $G = (N, A)$ be a directed graph of potential arcs to be constructed. We need to select a subset of arcs from A to route commodities $k \in [K]$. Each commodity k goes from source s_k to target t_k with volume v_k . Each arc $a \in A$ has capacity c_a and construction cost f_a . The goal is to design a network at minimum cost that is able to route all the flow.

Let x_a be a binary variable equal to 1 if arc a is constructed. Let y_a^k denote the amount of commodity k flowing through a . The NDP can be formulated as:

$$\min_{x, y} \sum_a f_a x_a \quad \text{s.t.} \quad \begin{cases} \sum_{a \in \delta^+(i)} y_a^k - \sum_{a \in \delta^-(i)} y_a^k = (1_{i=s_k} - 1_{i=t_k}) v_k & \forall i \\ \sum_k y_a^k \leq c_a x_a & \forall a \\ x_a \in \{0, 1\} & \forall j \\ y_a^k \geq 0 & \forall (a, k) \end{cases} \quad (4)$$

There are two ways to look at this constraint matrix:

- Block structure in the commodities
- Block structure in the arcs

Parallel subproblems

Idea 5

In each of these problems, removing the coupling constraint would allow parallel solution of simpler subproblems. Can we use this insight to

1. Solve LPs faster?
2. Obtain a better relaxation for branch & bound when solving ILPs?

Indeed, the number of nodes in the branch & bound tree grows exponentially with the integrality gap, and some of the formulations above have really bad gaps.

Cutting stock relaxation

Exercise 6

What is the value of the linear relaxation of the cutting stock problem? How can we obtain it more easily?

II. Dantzig-Wolfe relaxation

II.1. Definition

Setting

Definition 7

Consider the following integer program with rational coefficients:

$$z_{\text{ILP}} = \max_x c^\top x \quad \text{s.t.} \quad \begin{cases} Ax \leq b & (\text{hard constraint}) \\ Cx \leq d & (\text{easy constraint}) \\ x \in \mathbb{Z}^n & (\text{integrality}) \end{cases} \quad (5)$$

and its continuous relaxation

$$z_{\text{LP}} = \max_x c^\top x \quad \text{s.t.} \quad \begin{cases} Ax \leq b \\ Cx \leq d \\ x \in \mathbb{R}^n \end{cases} \quad (6)$$

We define $S = \{x \in \mathbb{Z}^n : Ax \leq b, Cx \leq d\}$ the integer feasible set and $Q = \{x \in \mathbb{Z}^n : Cx \leq d\}$ the integer feasible set with the hard constraint relaxed. Furthermore, we assume that problems of the form $\max\{\tilde{c}^\top x : x \in Q\}$ are easy to solve.

Dantzig-Wolfe relaxation

Definition 8

The Dantzig-Wolfe relaxation of the ILP in Equation 5 is defined as

$$z_{\text{DW}} = \max_x c^\top x \quad \text{s.t.} \quad \begin{cases} Ax \leq b \\ x \in \text{conv}(Q) \end{cases} \quad (7)$$

Often better than the continuous relaxation

Proposition 9

We have $z_{\text{ILP}} \leq z_{\text{DW}} \leq z_{\text{LP}}$.

The feasible sets of these three problems satisfy

$$S = \{Ax \leq b\} \cap Q \subseteq \{Ax \leq b\} \cap \text{conv}(Q) \subseteq \{Ax \leq b\} \cap \{Cx \leq d\} \quad (8)$$

Not always

Exercise 10

In which cases is the Dantzig-Wolfe relaxation equal to the linear programming relaxation?

When $\text{conv}(Q) = \{Cx \leq d\}$, the Dantzig-Wolfe relaxation does not give a better bound than the linear programming relaxation (but it may still be easier to compute, see below).

II.2. Extended formulation

Extended formulation

Proposition 11

There exist v_k and r_h such that the Dantzig-Wolfe relaxation can be formulated as

$$\begin{aligned}
 \max_{\lambda, \mu} \quad & \sum_{k \in K} \lambda_k c^\top v_k + \sum_{h \in H} \mu_h c^\top r_h \\
 \text{s.t.} \quad & \sum_{k \in K} \lambda_k A v_k + \sum_{h \in H} \mu_h A r_h \leq b \\
 & \sum_{k \in K} \lambda_k = 1 \\
 & \lambda \in \mathbb{R}_+^K, \mu \in \mathbb{R}_+^H,
 \end{aligned} \tag{9}$$

By Meyer's theorem, $\text{conv}(Q)$ is a rational polyhedron. By the Minkowski-Weyl theorem, it can be written as the sum of a polytope and a finitely-generated cone.

Reformulation, general case

Remark 12

To obtain a reformulation of the original ILP, just add the following constraint to the DW relaxation:

$$\sum_{k \in K} \lambda_k A v_k + \sum_{h \in H} \mu_h A r_h \in \mathbb{Z}^n \tag{10}$$

Reformulation, simple case

Remark 13

When Q is bounded, we can also pick v_k to be all points in Q (not just vertices), and then a solution to Equation 7 is integral if $\lambda \in \{0, 1\}^K$.

II.3. Back to coupling constraints

Many practical problems look like this (considering the pure binary case for simplicity):

$$\begin{aligned}
 \max_{x \in \{0,1\}^{n \times p}} \quad & c_1^\top x_1 + c_2^\top x_2 + \dots + c_p^\top x_p \\
 \text{s.t.} \quad & B_1 x_1 \leq b \\
 & B_2 x_2 \leq d \\
 & B_p x_p \leq b_p \\
 & D_1 x_1 + D_2 x_2 + \dots + D_p x_p \leq d
 \end{aligned} \tag{11}$$

Block-diagonal problems

Proposition 14

Block-diagonal problems have a natural DW decomposition.

Let $Q_j = \{x_j \in \{0, 1\}^n : B_j x_j \leq b_j\}$, the Dantzig-Wolfe relaxation is given by

$$\begin{aligned}
\max_{\lambda} \quad & \sum_j \sum_{v \in Q_j} \lambda_v^j c_j^\top v \\
\text{s.t.} \quad & \sum_j \sum_{v \in Q_j} \lambda_v^j D_j v \leq d \\
& \sum_{v \in Q_j} \lambda_v^j = 1 \quad \forall j \\
& \lambda \in \mathbb{R}_+^{|Q| \times p}
\end{aligned} \tag{12}$$

Identical blocks

Remark 15

Subproblems can be aggregated.

With p identical blocks, this simplifies with $\lambda_v = \sum_j \lambda_v^j$ (which counts the number of block solutions equal to v):

$$\begin{aligned}
\max_{\lambda} \quad & \sum_{v \in Q} \lambda_v c^\top v \\
\text{s.t.} \quad & \sum_{v \in Q} \lambda_v D_j v \leq d \\
& \sum_{v \in Q} \lambda_v = p \\
& \lambda \in \mathbb{R}_+^{|Q|}
\end{aligned} \tag{13}$$

This formulation removes symmetry.

II.4. Examples

Coming up with a DW reformulation

Remark 16

Either use the formula from the polyhedron $\text{conv}(Q)$, or try to guess it by enumerating all solutions.

Generalized assignment – DW

Exercise 17

Write the Dantzig-Wolfe reformulation of Equation 1.

Let $Q_j = \{x_j \in \{0, 1\}^m : \sum_i t_{ij} x_{ij} \leq T_j\}$ be a binary knapsack set relative to one agent. The DW reformulation of Equation 1 is:

$$\begin{aligned}
\max_{\lambda} \quad & \sum_j \sum_{v \in Q_j} \lambda_v^j c_j^\top v \\
\text{s.t.} \quad & \sum_j \sum_{v \in Q_j} \lambda_v^j v_i \leq 1 \quad \forall i \\
& \sum_{v \in Q_j} \lambda_v^j = 1 \quad \forall j \\
& \lambda_v^j \in \{0, 1\} \quad \forall (j, v)
\end{aligned} \tag{14}$$

But we could also write it by enumerating possible sets of tasks associated with each agent.

Cutting stock – DW

Exercise 18

Write the Dantzig-Wolfe reformulation of Equation 2.

Let $Q = \{(k, f) \in \{0, 1\} \times \mathbb{N}^m : \sum_i w_i f_i \leq Wk\} = \{(0, 0)\} \cup \{(1, f) : f \in F\}$ be the union of all cutting formats. The DW reformulation of Equation 2 is

$$\begin{aligned}
\min_{\lambda} \quad & \sum_f x_f \\
\text{s.t.} \quad & \sum_f x_f f_i \geq b_i \\
& \sum_f x_f \leq p \\
& x_f \in \mathbb{N} \quad \forall f
\end{aligned} \tag{15}$$

where x_f counts the number of times format f is used. Note that $\sum_f x_f \leq p$ is redundant by definition of p .

Facility location – DW

Exercise 19

Write the Dantzig-Wolfe reformulation of Equation 3.

Let $Q = \{(x, y) \in \{0, 1\}^n \times \{0, 1\}^{m \times n} : y_{ij} \leq x_j \forall (i, j)\}$ be the set of client assignments for each facility. Each element of Q can be expressed as a non-empty set of clients s associated with each open facility j . The DW reformulation of Equation 3 is:

$$\begin{aligned}
\min_{\lambda} \quad & \sum_j f_j \sum_s \lambda_s^j + \sum_{i,j} \sum_{s \ni i} c_{ij} \lambda_s^j \\
\text{s.t.} \quad & \sum_j \sum_{s \ni i} \lambda_s^j = 1 \quad \forall i \\
& \sum_s \lambda_s^j \leq 1 \quad \forall j \\
& \lambda_s^j \in \{0, 1\} \quad \forall (j, s)
\end{aligned} \tag{16}$$

Note that we don't constrain the sum of λ_s^j to be 1 because we want to allow choosing no set of clients for a facility (i.e. not opening it). Alternatively, we could allow the empty set $s = \emptyset$ and adjust the opening price.

Network design – DW

Exercise 20

Write the Dantzig-Wolfe reformulation of Equation 4.

Let Q_k be the flow polyhedron associated with commodity k . It is bounded due to finite capacity constraints, and its extreme points are elementary paths from s_k to t_k (no extreme rays) Let P_k be the set of such paths. The DW reformulation of Equation 4 is:

$$\begin{aligned}
 \min_{\lambda} \quad & \sum_a x_a \\
 \text{s.t.} \quad & \sum_k \sum_{\substack{p \in P_k \\ p \ni a}} \lambda_p^k \leq c_a x_a \quad \forall a \\
 & \sum_{p \in P_k} \lambda_p^k = 1 \quad \forall k \\
 & \lambda_p^k \in \{0, 1\} \quad \forall (k, p) \\
 & x_a \in \{0, 1\} \quad \forall a
 \end{aligned} \tag{17}$$

We could also have enumerated every possible flow (a more natural first solution perhaps).

III. Column generation

III.1. Subsets of variables

Exponential size

Remark 21

The Dantzig-Wolfe relaxation in Equation 9 has an exponential number of variables (size of the Minkowski-Weyl representation of $\text{conv}(Q)$). It cannot be solved “as is” by existing solvers.

We describe an approach to tackle this exponential size by considering a few generators only.

The restricted Dantzig-Wolfe relaxation is defined by using a subset of vertices $K' \subset K$ and a subset of rays $H' \subset H$ in Equation 9:

$$\begin{aligned}
& \max_{\lambda, \mu} \quad \sum_{k \in K'} \lambda_k c^\top v_k + \sum_{h \in H'} \mu_h c^\top r_h \\
& \text{s.t.} \quad \sum_{k \in K'} \lambda_k A v_k + \sum_{h \in H'} \mu_h A r_h \leq b \\
& \quad \sum_{k \in K'} \lambda_k = 1 \\
& \quad \lambda \in \mathbb{R}_+^{K'}, \mu \in \mathbb{R}_+^{H'},
\end{aligned} \tag{18}$$

If it is unbounded, then Equation 9 is also unbounded. Otherwise, let $(\bar{\lambda}, \bar{\mu})$ be an optimal solution of Equation 18.

If $(\bar{\lambda}, \bar{\mu})$ is not optimal for Equation 9, the question is how to select new generator candidates to insert into K' and H' .

The dual of Equation 9 is given by

$$\begin{aligned}
& \min_{\sigma, \pi} \quad \sigma + \pi^\top b \\
& \text{s.t.} \quad (c - A^\top \pi)^\top v_k - \sigma \leq 0 \quad \forall k \in K \\
& \quad (c - A^\top \pi)^\top r_h \leq 0 \quad \forall h \in H \\
& \quad \pi \geq 0
\end{aligned} \tag{19}$$

In the unrestricted Dantzig-Wolfe relaxation, let $\sigma \in \mathbb{R}$ be the Lagrange multiplier associated with constraint $\sum_{k \in K} \lambda_k = 1$, and $\pi \geq 0$ the vector of Lagrange multipliers associated with $\sum_{k \in K} \lambda_k A v_k + \sum_{h \in H} \mu_h A r_h \leq b$. The Lagrangian writes

$$\begin{aligned}
\mathcal{L}(\lambda, \mu; \sigma, \pi) &= \sum_k \lambda_k c^\top v_k + \sum_h \mu_h c^\top r_h \\
&\quad + \sigma \left(1 - \sum_k \lambda_k \right) \\
&\quad + \pi^\top \left(b - \sum_k \lambda_k A v_k - \sum_h \mu_h A r_h \right) \\
\mathcal{L}(\lambda, \mu; \sigma, \pi) &= \sum_k \lambda_k (c^\top v_k - \pi^\top A v_k - \sigma) \\
&\quad + \sum_h \mu_h (c^\top r_h - \pi^\top A r_h) \\
&\quad + \sigma + \pi^\top b
\end{aligned} \tag{20}$$

And thus the dual is $\min_{\sigma, \pi} \max_{\lambda, \mu} \mathcal{L}(\lambda, \mu; \sigma, \pi)$, which boils down to

$$\begin{aligned} \min_{\sigma, \pi} \quad & \sigma + \pi^\top b \\ \text{s.t.} \quad & c^\top v_k - \pi^\top A v_k - \sigma \leq 0 \quad \forall k \\ & c^\top r_h - \pi^\top A r_h \leq 0 \quad \forall h \\ & \pi \geq 0 \end{aligned} \tag{21}$$

Dual variables are shadow prices

Idea 24

Let $(\bar{\sigma}, \bar{\pi})$ be a dual optimal solution associated with $(\bar{\lambda}, \bar{\mu})$ for Equation 18. The dual variables tell us whether it is worth adding new generators to K' and H' .

III.2. Pricing

Finding new generators

Proposition 25

For $k \in K$, let $\bar{c}_k = (c - A^\top \bar{\pi})^\top v_k - \bar{\sigma}$ be the reduced cost associated with vertex v_k . For $h \in H$, let $\bar{c}_h = (c - A^\top \bar{\pi})^\top r_h$ be the reduced cost associated with ray r_h . If all $\bar{c}_k$ are zero and all $\bar{c}_h$ are zero, then $(\bar{\lambda}, \bar{\mu})$ is an optimal solution to the full relaxation Equation 18. Otherwise, generators with positive reduced costs can be added to the main problem to improve the solution.

Indeed, if $((\bar{\lambda}, \bar{\mu}), (\bar{\sigma}, \bar{\pi}))$ is a primal-dual optimal pair for Equation 18, then they satisfy complementary slackness. Furthermore, $(\bar{\lambda}, \bar{\mu})$ is feasible for the full problem Equation 9. Thus, $(\bar{\lambda}, \bar{\mu})$ is optimal for Equation 9 as well if and only if $(\bar{\sigma}, \bar{\pi})$ is feasible for its dual Equation 19. If it is not feasible, there must be a violated constraint, and we can add it to the dual, which means adding the corresponding variable to the primal.

Simplex dual solution

Exercise 26

Given an optimal basis B in the simplex algorithm for a standard-form LP $\max\{c^\top x : Ax = b, x \geq 0\}$, how does one retrieve a dual optimal solution?

The dual problem is $\min\{b^\top y : A^\top y \geq c\}$. At the end of the simplex, the reduced costs are negative:

$$r_N = c_N - A_N^\top A_B^{-\top} c_B \leq 0 \tag{22}$$

This gives us a natural candidate for the dual optimal solution: $y^* = A_B^{-\top} c_B$. We can also check that $b^\top y^* = c^\top x^*$.

Pricing subproblem

Definition 27

The pricing subproblem is the optimization problem

$$\zeta = -\bar{\sigma} + \max_{x \in Q} (c - A^\top \bar{\pi})^\top x \quad (23)$$

There can be three situations:

- The problem Equation 23 is unbounded if and only if there is an extreme ray r_h such that $(c - A^\top \bar{\pi})^\top r_h > 0$, i.e. with positive reduced cost.
- If it is bounded and $\eta > 0$, then there is a vertex v_k with positive reduced cost.
- If it is bounded and $\eta \leq 0$, no variable has a positive reduced cost.

III.3. Full algorithm

Column generation for large MILPs

Algorithm 28

1. Start with variable subsets K' and H' .
2. Solve the restricted main problem (Equation 18), obtain $(\bar{\lambda}, \bar{\mu})$ and $(\bar{\sigma}, \bar{\pi})$.
3. Solve the pricing subproblem (Equation 23), and depending on its result, either terminate or add variables to K' / H' and loop.

To control convergence, use the optimum of the pricing problem.

Gap control

Proposition 29

Let $(\bar{\lambda}, \bar{\mu})$ be a feasible solution to the restricted main problem (no extreme rays), and $(\bar{\sigma}, \bar{\pi})$ be an associated dual solution. Let ζ be the value of the pricing subproblem, which we assume finite and > 0 . Then the optimum of the full main problem has value in $[c^\top \bar{x}, c^\top \bar{x} + \zeta]$.

Indeed, if $\zeta > 0$, then ζ corresponds to the maximum violation of a dual constraint in Equation 19 by a vertex of Q . This means that $(\bar{\sigma} + \zeta, \bar{\pi})$ is dual-feasible, with dual objective value $\bar{\sigma} + \zeta + \bar{\pi}^\top b = c^\top \bar{x} + \zeta$.

III.4. Examples

Generalized assignment – pricing

Exercise 30

Identify the pricing subproblem of Equation 14. How would you solve it?

Binary knapsack problem, possibly with negative costs. Solvable by dynamic programming.

Cutting stock – pricing

Exercise 31

Identify the pricing subproblem of Equation 15. How would you solve it?

Integer knapsack problem, possibly with negative costs. Solvable by dynamic programming.

Facility location – pricing

Exercise 32

Identify the pricing subproblem of Equation 16. How would you solve it?

Unconstrained binary linear program, decomposes by customer: only add the ones who are beneficial. Capacity constraints would couple them.

Network design – pricing

Exercise 33

Identify the pricing subproblem of Equation 17. How would you solve it?

Shortest path problem, possibly with negative weights. Solvable with Bellman-Ford.

IV. Branch and price

Branch & price

Algorithm 34

The branch & price algorithm is a branch & bound where at each node, the relaxation is given by Dantzig-Wolfe.

IV.1. Branching techniques

Adding branching inequalities

Remark 35

It is better to add branching inequalities to the main problem than the pricing problem.

Otherwise one may disturb the specific structure that makes pricing easy.

Variable to branch on

Remark 36

It is better to branch on the original variables x as opposed to the reformulation variables λ .

Two reasons:

- Fixing $\lambda_k = 1$ means separating just one vertex from the rest of the polytope, not a very useful split.
- Fixing $\lambda_k = 0$ doesn't mean we won't try to include v_k further down in the pricing step, which means we'd have to look for the second-best solution of pricing.

IV.2. Implementation tricks

Suboptimal pricing

Remark 37

Still works as long as the columns have positive reduced cost.

No need to fully resolve pricing, except at the very end to check optimality.

Column addition or removal

Remark 38

Nothing stops us from adding several columns at once to the main problem. Conversely, we can also clean up when it gets too big.

Useful when the solver naturally produces several solutions.

Warm starting

Remark 39

One can restart the primal simplex from the previous solution, since adding a variable doesn't affect primal feasibility.

How does this differ from branch & cut? Branch & cut uses the dual simplex.

Double block angular structure

Exercise 40

How to handle the case with binding constraints?

Split the variables.

IV.3. Flow subproblems

A very common case.

Vertex and ray pricing

Exercise 41

Which algorithm can be used to solve both pricing subproblems for a flow?

Bellman-Ford will either identify a negative-cost cycle or find an elementary shortest path. In special cases we can do even better, e.g. for acyclic graphs (topological sort).

IV.4. Getting an integer solution

There are some heuristics to get a suboptimal integer solution from the output of the DW decomposition.

MIP-ifying

Remark 42

Idea: run column generation then solve the restricted main problem one last time as an integer program (forcing vertex coefficients to be zero or one). Then we get a feasible solution to the original MILP.

IV.5. Take-home messages

When to use DW?

- Not appropriate for general problems without structure.
- When the pricing subproblems are independent (block-diagonal structure).
- When pricing can be done with a dedicated solver.
- Especially relevant when pricing is NP-hard in theory (perfect formulations don't give better bounds) but well-solved in practice. Weakly NP-hard is a good balance (knapsack).

Bibliography

- [1] M. Conforti, G. Cornuéjols, and G. Zambelli, *Integer Programming*, 1st ed. 2014., no. 271. in Graduate Texts in Mathematics. Cham: Springer International Publishing : Imprint: Springer, 2014. doi: 10.1007/978-3-319-11008-0.
- [2] E. Uchoa, A. Pessoa, and L. Moreno, “Optimizing with Column Generation: Advanced Branch-Cut-and-Price Algorithms (Part I),” technical report L-2024–3, Aug. 2024. [Online]. Available: <https://optimizingwithcolumnngeneration.github.io/>